COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/30/2026	**DATE SUBMITTED:**	08/03/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

HANDS ON ACTIVITY #4 **STACKS**

1. PROCEDURE OUTPUT

ILO B #2:

stackList_LL.h

```cpp
#ifndef STACKLIST_LL_H
#define STACKLIST_LL_H
#include <iostream>

//class node with dot and next members
template <typename T>
class Node{
    public:
    T data;
    Node<T>* next = nullptr;
};
//create a pointer/reference tip that always point to the current top node
//global
template <typename T>
Node<T>* head = nullptr; //since its a linked list

//push operation
template <typename T>
void push(T newData){
    //create a new node
    Node<T>* newNode = new Node<T>;
    //assign the data to the new node
    newNode->data = newData;
    //point the newNode next to the current top
    newNode->next = head<T>;
    //assign the new top to the newNode
    head<T> = newNode;
}
//isEmpty operation
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

//pop operation
template <typename T>
T pop(){
    //if the stack is empty error catch:
    if(isEmpty<T>()){
        std::cout << "Stack Underflow." << std::endl;
    }
    //storing a current top node to a temporary variable
    Node<T>* temp = head<T>;
    //store the data to a new variable
    T tempVal = temp->data;
    //move the head of the node to a temporary variable
    head<T> = head<T>->next;
    //free up the memory
```

```
49      delete temp;
50      //return the variable data
51      return tempVal;
52  }
53  //top
54  template <typename T>
55  void Top(){
56      //what if the stack is empty???
57      if(isEmpty<T>()) return;
58
59      std::cout <<"The top is: " << head<T>->data << std::endl;
60  }
61  //display all
62  template <typename T>
63  void stackTraversalDisplayAll(){
64      //error checking
65      if(isEmpty<T>()) return;
66      //loop
67      do {
68          std::cout << "Displayed stack: " << head<T>->data << std::endl;
69          head<T> = head<T>->next;
70          } while (head<T> != nullptr);
71      std::cout << std::endl;
72  }
73  #endif
```

Analysis:

First in the class node, the code used public T data so it would be used as template and Node<T>* because we are going to use the pointer as the storage for data and Node<T>* next so it would be pointed in the next node. For the push operation, the code has given a template with the void function push with its values being the new data (T newData) inside of the function we create the a new node, assign that data into the new node, point it to the top (newNode to top) and assigned it now at the top (new) to the new node. For the error checking which if the stack is empty, it returns as a nullptr. For the pop function, asides from the error checking which it would print as “Stack Underflow” indicating there is no stack in the pop function. The rest is putting the variable into a temporary variable, stores it into a new variable and moves it to the head and once it's done the temporary variable will be deleted and returns it into the actual variable. For the top function, asides from the addition of the bool operation, it would print if what stack is the top. For the display all, aside from the error checking, I made a do-while loop that displays the stack, displaying the data from head<T> in the node.

main.cpp

```cpp
#include<iostream>
#include "stackList_LL.h"

int main(){
    std::cout<<"---------- Pushing 'I' 'E' 'E' 'E'-------- \n";
    //pushing
    push('I');
    push('E');
    push('E');
    push('E');
    //top
    Top<char>();
    //display all
    stackTraversalDisplayAll<char>();
    //pop and print each element
    std::cout<<"---------- Popping 'I' 'E' 'E' 'E'-------- \n";
    while(isEmpty<char>()){
        std::cout<< "Popping: "<< pop<char>() << std::endl;
    }
    //test stack underflow
    pop<char>;
    stackTraversalDisplayAll<char>();

    return 0;
}
```

Analysis:

In the header file we put the stack list file then back to the main function we push the letters ('I', 'E', 'E', 'E') then we display what stack is the top (which is 'E') and also display the stack. In the pop function, the code tries to print the pop but since we don't put anything there it prints the stack underflow instead..

Output of the Code:

```
---------- Pushing 'I' 'E' 'E' 'E'--------
The top is: E
Displayed stack: E
Displayed stack: E
Displayed stack: E
Displayed stack: I

---------- Popping 'I' 'E' 'E' 'E'--------
Popping: Stack Underflow.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C (Array):

Code:

supplementaryArrays_LL.h:

```
#ifndef SUPPLEMENTARYARRAYS_LL_H
#define SUPPLEMENTARYARRAYS_LL_H
#include <iostream>

//For boolean symbols:
bool matchPairSymbols(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

//Class for Stack Array:
class arrayStack {
private:
    char arr[100];
    int top;

public:
    arrayStack() {
        top = -1;
    }

    bool isEmpty() {
        return top == -1;
    }

    bool isFull() {
        return top == 100 - 1;
    }

    void push(char ch) {
        if (!isFull()) {
            arr[++top] = ch;
```

```
34          }
35      }
36
37      char pop() {
38          if (!isEmpty()) {
39              return arr[top--];
40          }
41          return 0;
42      }
43  };
44
45  //checking if the pair is matched
46  bool matchBalanceSymbols(std::string input) {
47      arrayStack stack;
48      for (char ch: input) {
49          if (ch == '(' || ch == '{' || ch == '[') {
50              stack.push(ch);
51          }
52
53          else if (ch == ')' || ch == '}' || ch == ']') {
54              if (stack.isEmpty())
55                  return false;
56
57              char top = stack.pop();
58
59              if (!matchPairSymbols(top, ch))
60                  return false;
61          }
62      }
63      return stack.isEmpty();
64  }
65  #endif
```

supplementary.cpp:

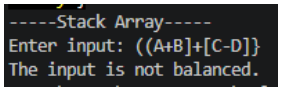
```cpp
#include <iostream>
#include "supplementaryArrays_LL.h"

int main() {

    std::cout << "-----Stack Array-----" << std::endl;
    std::cout << "Enter input: ";
    std::string input;
    getline(std::cin, input);

    if (matchBalanceSymbols(input)){
        std::cout << "The input is balanced." << std::endl;
     }else{
        std::cout << "The input is not balanced." << std::endl;
    }
    return 0;
}
```

Expression	Valid (Y/N)	Output	Analysis
(A+B)+(C-D)	Yes	-----Stack Array----- Enter input: (A+B)+(C-D) The input is balanced.	It is balanced since both opening and closing delimiters are the same. It reads '(' so it was put in the stack and once it finds ')' it becomes empty. This is also the same with (C-D).
((A+B)+(C-D)	No	-----Stack Array----- Enter input: ((A+B)+(C-D) The input is not balanced.	It is not balanced since there is still an extra '(' and it doesn't have a closing delimiter. It returns false since the stack was not empty.
((A+B)+[C-D])	Yes	-----Stack Array----- Enter input: ((A+B)+[C-D]) The input is balanced.	It is balanced since similar to the first expression it has both an opening and closing

			delimiters even though C-D used brackets. The stacks return empty regardless.
((A+B]+[C-D]}	**No**		**It is not balanced since there are a lot of mixed delimiters that neither have a closing delimiter nor an opening delimiter. The stack was not empty so it returns false.**

Analysis:

For supplementaryArrays_LL.h:

I added first the header guards which are #ifndef, #define, #endif before adding the header file #include <iostream>. I added the boolean function of the matching delimiters ((), {}, []), it returns true if it matches correctly. Next is, I added the class for the arrayStack, the max capacity of the array is 100 and the int top so it stores the top element of the stack. I added the usual functions of the stack which are isEmpty, push and pop and also added isFull since the valid array index is 99. The push function has an increment (arr[++top}) that will push it to the top of the stack and the pop function has a decrement (arr[top--]) that will remove the top of the stack. Next is the matchBalanceSymbols(), this function checks if the delimiters matched or not, it detects true if both opening and closing are matched and false if there is a missing or extra.

For supplementary.cpp:

I added first the header file #include <iostream> and “supplementaryArray_LL.h” to connect it to the file. I first added “Enter Input” before I interacted with it. I used std::string input since the expressions given contain characters like letters, symbols and delimiters. I used the getLine so the whole expression can be stored and to know if the expression given was balanced or not. I also used an if/else statement whether the symbols are balanced or not.

ILO C (Linked List):

Code:
supplementaryLL_LL.h:

```cpp
#ifndef SUPPLEMENTARYLL_LL_H
#define SUPPLEMENTARYLL_LL_H
#include <iostream>

//For boolean symbols:
bool matchPairSymbols(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

//Class for Node:
class Node {
    public:
    char data;
    Node* next;

    Node(char LL){
        data = LL;
        next = NULL;
    }
};

//Class for Stack Linked List:
class LLStack {
private:
    Node* top;
public:
    LLStack() {
        top = NULL;
    }

    bool isEmpty() {
```

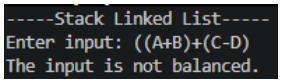
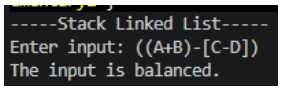
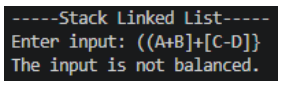
```
33      bool isEmpty() {
34          return top == NULL;
35      }
36
37      void push(char ch) {
38          Node* newNode = new Node(ch);
39          newNode->next = top;
40          top = newNode;
41      }
42
43      char pop() {
44          if (isEmpty()) {
45              return 0;
46          }
47
48          Node* temp = top;
49          char value = temp->data;
50          top = top->next;
51
52          delete temp;
53
54          return value;
55      }
56  };
57
58  //checking if the pair is matched
59  bool matchBalanceSymbols(std::string input) {
60      LLStack stack;
61      for (char ch: input) {
62          if (ch == '(' || ch == '{' || ch == '[') {
```

```
63              stack.push(ch);
64          }
65
66          else if (ch == ')' || ch == '}' || ch == ']') {
67              if (stack.isEmpty())
68                  return false;
69
70              char top = stack.pop();
71
72              if (!matchPairSymbols(top, ch))
73                  return false;
74          }
75      }
76      return stack.isEmpty();
77  }
78  #endif
```

supplementary2.cpp:

```
1   #include <iostream>
2   #include "supplementaryLL_LL.h"
3
4   int main(){
5
6      std::cout << "-----Stack Linked List-----" << std::endl;
7      std::cout << "Enter input: ";
8      std::string input;
9      getline(std::cin, input);
10
11     if (matchBalanceSymbols(input)){
12        std::cout << "The input is balanced." << std::endl;
13     }else{
14        std::cout << "The input is not balanced." << std::endl;
15     }
16
17     return 0;
18  }
```

Expression	Valid (Y/N)	Output	Analysis
(A+B)+(C-D)	Yes	-----Stack Linked List----- Enter input: (A+B)+(C-D) The input is balanced.	It is balanced since both opening and closing delimiters are the same. It reads '(' so it was put in the stack

			and once it finds ‘)’ it becomes empty. This is also the same with (C-D).
((A+B)+(C-D)	No		It is not balanced since there is still an extra ‘(‘ and it doesn’t have a closing delimiter. It returns false since the stack was not empty.
((A+B)+[C-D])	Yes		It is balanced since similar to the first expression it has both an opening and closing delimiters even though C-D used brackets. The stacks return empty regardless.
((A+B]+[C-D]}	No		It is not balanced since there are a lot of mixed delimiters that neither have a closing delimiter nor an opening delimiter. The stack was not empty so it returns false.

Analysis:

For supplementaryLL_LL.h:

I added first the header guards which are #ifndef, #define, #endif before adding the header file #include <iostream>. I added the boolean function of the matching delimiters ((), {}, []), it returns true if it matches correctly. Next is, I added the class for the Node, this will represent as the one element that would be connected in the linked list, we store data inside the node and Node* next to store its address to the stack. I also added the constructor so it will initialize a new node (Node (char LL)). Next is the class for LLStack, in the private I added Node* top this will store the address of the node on the top of the stack. In the public, I put the top of the stack as null this will create an empty stack. In the isEmpty function, it checks if the stack is empty (return top == NULL) this is also error checking. In the push function, since it's a linked list stack, it creates a new node and connects it to the top of the node instead of pushing the stack. In the pop function, it removes the latest element in the node in the stack since it's a linked list. Similar to the last code, I added the matchBalanceSymbols() to detect whether both opening and closing delimiters are the same or not.

For supplementary2.cpp:

I added first the header file #include <iostream> and “supplementaryLL_LL.h” to connect it to the file. I first added “Enter Input” before I interacted with it. I used std::string input since the expressions given contain characters like letters, symbols and delimiters. I used the getLine so the whole expression can be stored and to know if the expression given was balanced or not. I also used an if/else statement whether the symbols are balanced or not.

Conclusion:

I was able to learn how stack works and how their functions work. I also learned that stacks with arrays and linked lists can be used although I didn’t do the optional activity. Overall, I still need to absorb most of the activity even though I finished the activity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT to know how to explain the concepts in the code as well as generate ideas for the code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.